FlightLens: US Flight Performance Data Platform

Project Proposal

Item	Details
Program	Digital Egypt Pioneers Initiative (DEPI), Round 5
Track	AI & Data Science – Microsoft Data Engineer
Training provider	Skills Dynamix
Group	Group 3 · ALX5_AIS5_S6
Instructor	Ahmed Essam Azab
Team	Nepal Adel Ali, Asmaa Adel Ali, Omar Emad Hamdy, Abdelrahman Mahmoud Mohamed, Mosaab Mohamed Mohamed

Executive summary

FlightLens is an end-to-end data pipeline that turns raw US flight records into clean, tested, analysis-ready data, so airline and airport managers can see where, when and why flights are delayed or cancelled.

The source is the official on-time database of the US Bureau of Transportation Statistics (BTS): every domestic flight from July to December 2025, 3,554,943 rows. The pipeline downloads it month by month, cleans it with PySpark through Bronze, Silver and Gold layers, and loads a star schema into PostgreSQL. A Power BI dashboard, a delay-prediction model and a GenAI question-answering assistant sit on top.

The whole platform runs on free, open-source software in Docker, so any team member can start it with one command and no cloud subscription. The design maps one-to-one to Microsoft Azure services, so it can move to the cloud later without redesign.

Problem statement

BTS publishes every US domestic flight, but in a form analysts cannot use directly. Data arrives as one large file per month. Times are local `hhmm` numbers, delay-cause fields are blank for most rows, and codes need lookup tables. Every analyst repeats the same cleaning and gets slightly different answers.

FlightLens cleans the data once, tests it, and publishes it as one trusted source that answers these questions:

#	Business question	Delivered by
Q1	Which airports have the most delays and cancellations?	Dashboard
Q2	What causes delays: carrier, weather, air traffic, security or a late incoming aircraft?	Dashboard
Q3	Do delays rise in certain months, weekdays or hours?	Dashboard
Q4	Which carriers have the highest delay and cancellation rates?	Dashboard
Q5	Which routes are delayed most often?	Dashboard

Q6	Are long-distance flights more likely to be delayed?	Dashboard
Q7	Will a given scheduled flight arrive 15+ minutes late?	ML model
Q8	Can a manager get these answers by asking in plain English?	GenAI assistant

Objectives and scope

The scope is split into three tiers so the team delivers a complete system first and adds features only as time allows.

Tier	Features	Commitment
MVP	Automated monthly download; Bronze, Silver and Gold layers; data-quality checks; star schema in PostgreSQL; Power BI dashboard (Q1–Q6); Airflow orchestration; Docker setup	Must be finished to pass
Planned	Delay-prediction model with API (Q7); GenAI assistant (Q8); CI/CD with GitHub Actions	Planned in the schedule; dropped only if the MVP slips
Stretch	Hourly weather enrichment; simulated real-time stream with Kafka	Only if time remains in weeks 21–22

In scope: US domestic flights reported to BTS, July to December 2025 (3,554,943 flights), covering the summer travel peak and the holiday season.

Out of scope: international flights, airlines outside BTS reporting, fares and passenger counts, and real production feeds.

Requirements

Stakeholders, user stories, use cases, and the functional and non-functional requirements are in a separate document, *FlightLens: Requirements Specification*.

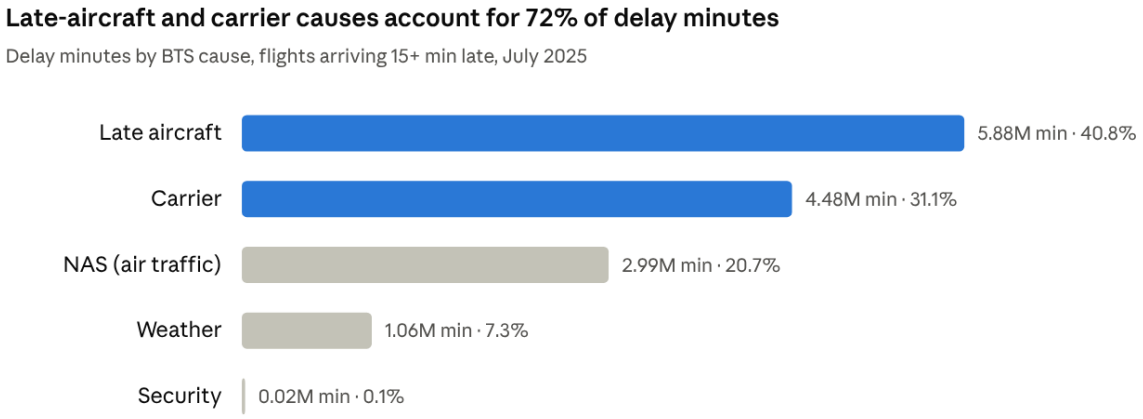
Data source

The data is the official BTS Airline On-Time Performance database of the US Department of Transportation. US carriers must report every domestic flight under federal rule 14 CFR 234, so the records are real and complete, not samples or estimates.

Item	Value
Publisher	Bureau of Transportation Statistics, US Department of Transportation
Coverage	US domestic flights of reporting carriers, July to December 2025 (6 months)
Volume	3,554,943 flights, 1 Jul to 31 Dec 2025
Format	One ZIP/CSV file per month, ~110 columns; 32 kept
Granularity	One row per flight
Acquisition	`download_bts.py` downloads each month automatically; July–December

	2025 downloaded and saved as one CSV file per month with the 32 selected columns
Licence	US government public data, free to use

The 32 columns cover the flight (date, carrier, flight and tail number), route (origin, destination, states, distance), schedule and actual times, delay minutes and 15-minute flags, five delay causes, cancellation, diversion, and taxi and air time.

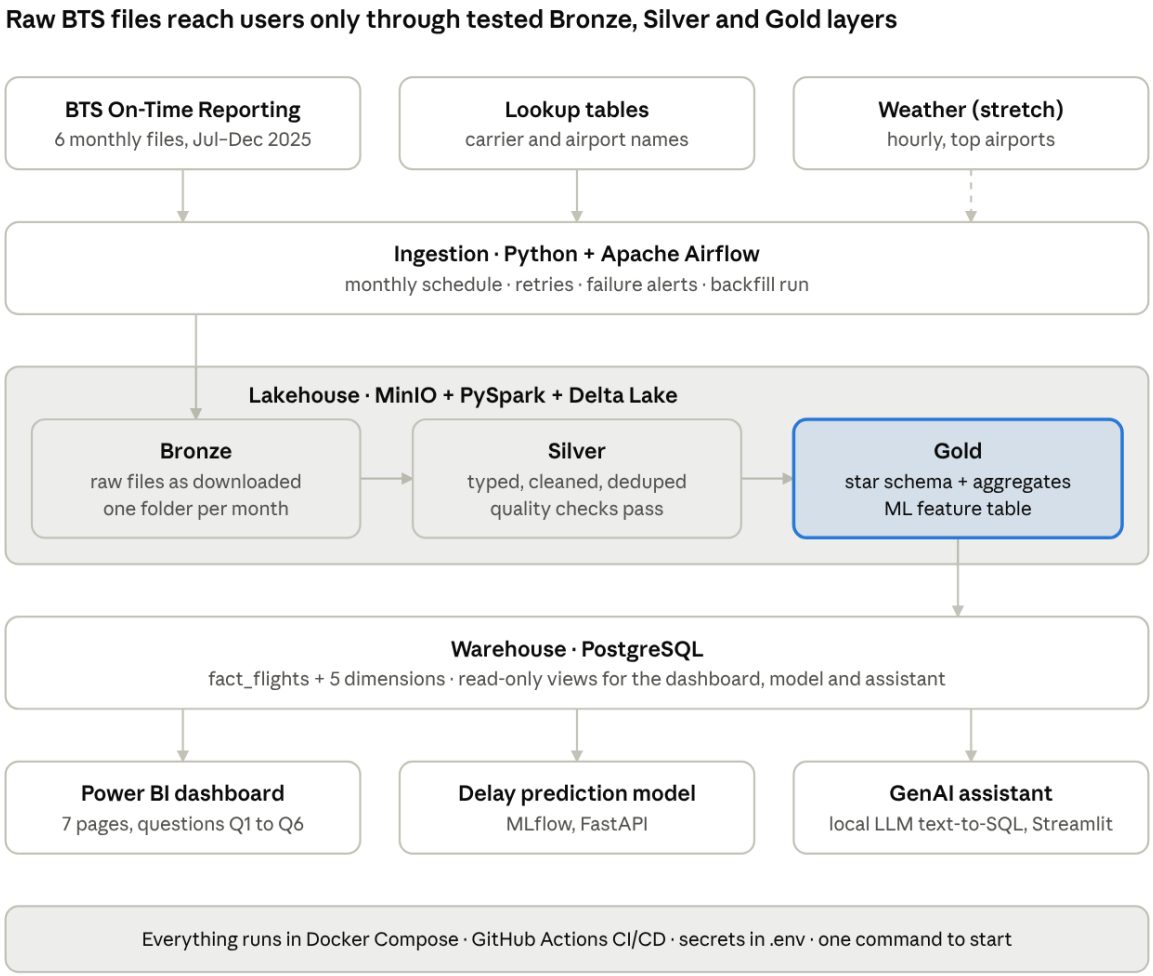


BTS On-Time Reporting · 631,428 flights, July 2025

Known data issues the pipeline must fix: times are local `hhmm` numbers including `2400`; cancelled flights have no actual times; delay-cause columns are blank in 72% of July rows because BTS fills them only for 15+ minute delays; diverted flights need their own handling; overnight flights arrive the next day.

Solution architecture

FlightLens follows the medallion pattern: Airflow lands raw files, PySpark refines them through three Delta Lake layers, and PostgreSQL serves the Gold layer to every consumer.



FlightLens architecture · open-source stack in Docker

Each hop between layers is gated by quality checks. A month is loaded once; re-running it replaces that month instead of adding duplicates.

Technology stack

Every component is free and open source, and each one has a direct Microsoft Azure equivalent taught in the course.

Layer	Tool	Purpose	Azure equivalent
Ingestion	Python (`requests`, `pandas`)	Download monthly BTS files	Data Factory copy activity
Orchestration	Apache Airflow	Schedule and monitor the monthly pipeline, retries, alerts	Azure Data Factory
Storage (lake)	MinIO; CSV in Bronze, Delta	Bronze, Silver and Gold	ADLS Gen2

	Lake in Silver and Gold	layers	
Processing	PySpark + Delta Lake	Cleaning and transforming 3.55M rows	Azure Databricks
Data quality	Great Expectations	Automated checks between layers	Databricks expectations
Warehouse	PostgreSQL	Star schema and SQL views	Azure Synapse Analytics
BI	Power BI Desktop	Dashboard	Power BI Service
ML	scikit-learn, LightGBM, MLflow, FastAPI	Train, track and serve the delay model	Azure Machine Learning
GenAI	Ollama (local LLM) or Gemini API free tier, Streamlit	Plain-English questions → SQL	Azure OpenAI
DevOps	Docker Compose, GitHub, GitHub Actions	One-command setup, version control, CI/CD	Azure DevOps, Container Apps

Optional: the Spark jobs can also be run on Databricks Free Edition to show the same code working in the cloud.

Data model and data quality

The Gold layer is a star schema with one row per flight in the fact table, so every business question becomes a simple join and group-by.

- **Fact:** `fact_flights` (one scheduled flight: delays, flags, delay causes, distance, times).
- **Dimensions:** `dim_date`, `dim_time` (hour blocks), `dim_carrier`, `dim_airport` (used for both origin and destination), `dim_route`.
- **Aggregate:** `agg_daily_airport_kpis` for fast dashboard pages.

The full ER diagram and physical schema are design-phase deliverables (week 14).

Every run checks the data before promoting it to the next layer: schema and types, one row per flight (no duplicates), valid times and codes, cancelled flights without actual times, and every fact key present in its dimension. Failing rows go to a quarantine folder instead of Gold, and results are shown on a data-health page.

AI and machine learning

Two AI features use only the trusted Gold data, which shows the pipeline's value.

1. Delay-prediction model (Q7). A classifier predicts whether a scheduled flight will arrive 15+ minutes late.

- **Features known before departure only:** carrier, origin, destination, distance, scheduled hour, weekday, month, holiday flag, and the route's and carrier's historical on-time rate from earlier months.

- **Leakage guard:** actual departure time, departure delay, taxi time and delay causes are excluded, because they are only known after the flight.
- **Validation:** time-based split: train July–October, validate November, test December 2025.
- **Models and metrics:** logistic regression baseline, then LightGBM; ROC-AUC and PR-AUC, since only about 23% of completed flights are late and accuracy alone would mislead.
- **Serving:** experiments tracked in MLflow; best model served through a FastAPI endpoint.

2. GenAI assistant (Q8). A chat page where a manager asks, for example, "Which 5 airports had the worst on-time rate in December?". A local LLM (Ollama) or the Gemini API free tier turns the question into SQL, using the schema and data dictionary as context. The SQL runs on a read-only PostgreSQL user, and the page shows the answer, the table and the SQL. A 30-question test set with known answers measures accuracy.

Analytics dashboard

A Power BI Desktop report connected to PostgreSQL answers Q1–Q6, with date, carrier, airport and state filters on every page.

Page	Shows	Questions
Overview	Flights, on-time %, cancellation %, average delay, monthly trend	Q3
Airports	US map of airports by on-time %; 10 worst airports	Q1
Carriers	On-time and cancellation rate per carrier	Q4
Delay causes	Delay minutes by cause, by carrier and airport	Q2
Routes and distance	20 worst routes (min. 100 flights); delay rate by distance group	Q5, Q6
Time patterns	Delay rate by hour × weekday heatmap	Q3
Data health	Last load, rows loaded, quality-check results	Trust in the data

Deployment and DevOps

The whole platform starts with one command, `docker compose up`, on any team member's laptop, with no cloud account needed.

- **Containers:** Airflow, Spark, MinIO, PostgreSQL, MLflow, the prediction API and the assistant each run in their own container.
- **Version control:** one GitHub repository, a feature branch per task, pull requests reviewed by a second member.
- **CI/CD:** GitHub Actions runs unit tests (pytest) on a small data sample and builds the Docker images on every pull request.
- **Configuration:** passwords and API keys live in an `.env` file that is never committed.
- **Demo:** the final demo runs live on the strongest laptop; a recorded video is the backup.
- **Hardware:** at least one laptop with 16 GB RAM runs the full stack; others can run only the services they work on.

Team and roles

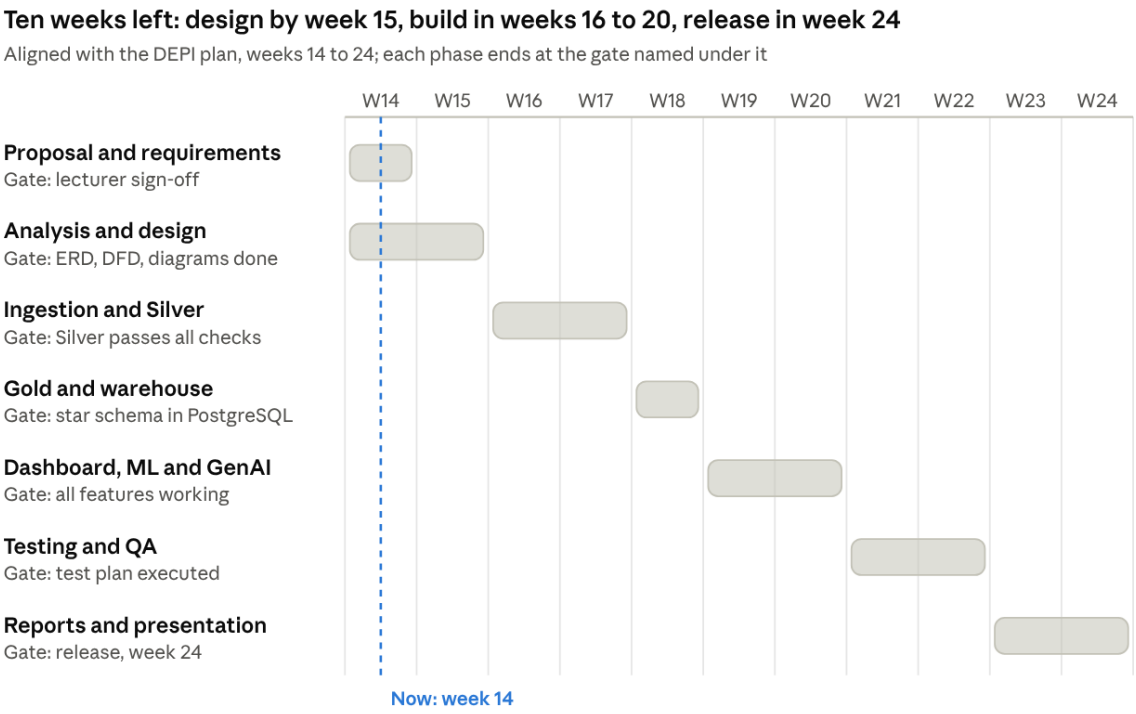
Each member owns one layer end to end, plus the design documents for that layer.

Member	Role	Owns in implementation	Owns in design (weeks 14–15)
Asmaa Adel Ali	Data Ingestion Engineer	Download script, Airflow DAGs, Bronze layer, scheduling, retries and alerts	Data flow diagram, activity diagram
Mosaab Mohamed Mohamed	Big Data Engineer (Spark)	PySpark cleaning, Silver layer, Delta Lake, data-quality checks	Class diagram, state diagram, data-quality test plan
Nepal Adel Ali	Analytics Engineer	Star schema, Gold layer, PostgreSQL views, Power BI dashboard	ER diagram, logical and physical schema, dashboard wireframes
Omar Emad Hamdy	ML Engineer	Feature table, model training and evaluation, MLflow, prediction API	Sequence diagram (prediction), API documentation
Abdelrahman Mahmoud Mohamed	GenAI & DevOps Engineer	GenAI assistant, Docker Compose, GitHub Actions, README	Sequence diagram (assistant), deployment and component diagrams

All members write tests for their own part, contribute to the user manual, and present together. Mosaab's Silver layer is the critical path, so Nepal and Omar start on the July sample while it is being built.

Timeline

The team is in week 14 with 10 weeks to release: proposal and requirements this week, design by week 15, build in weeks 16–20, testing in weeks 21–22, and release with the final presentation in week 24.



Project plan · weeks 14 to 24, aligned with the DEPI project guideline

Nepal and Omar prototype on the July sample during weeks 16–17, so they are not blocked while the Silver layer is built.

Deliverables

Each document the DEPI guideline requires is mapped to what FlightLens delivers and when.

DEPI phase	Required artifact	FlightLens deliverable	Owner	Week
1. Planning	Proposal, Gantt plan, roles, risks, KPIs	This document	All	14
3. Requirements	Stakeholders, user stories, use cases, FR / NFR	Requirements Specification	All	14
4. Design	Software architecture	Medallion pipeline (diagram in this document)	All	14
4. Design	ER diagram, logical and physical schema	Star schema with keys, types, indexes	Nepal	14
4. Design	Data flow diagram	BTS → Bronze → Silver → Gold → PostgreSQL → consumers	Asmaa	14
4. Design	Activity diagram	Pipeline run with quality gate and quarantine branch	Asmaa	15

4. Design	State diagram	Pipeline run states; flight status (scheduled → departed → arrived / cancelled / diverted)	Mosaab	15
4. Design	Class diagram	Python package: downloader, transformer, validator, loader	Mosaab	15
4. Design	Sequence diagrams	Prediction request; assistant question	Omar, Abdelrahman	15
4. Design	Wireframes, UI guidelines	Dashboard pages and assistant chat page	Nepal	15
4. Design	Tech stack, deployment and component diagrams	Docker containers and their dependencies	Abdelrahman	15
5. Implementation	Source code, GitHub, branching, CI/CD, README	Repository with pull requests and GitHub Actions	All	16–20
5. Implementation	API documentation	FastAPI auto-generated docs	Omar	19–20
6. Testing	Test plan, automated tests, bug reports	pytest suite, data-quality results, GitHub issues	All	21–22
7. Final	User manual, technical documentation, presentation, demo video	Guides, system docs, slides, 5-minute video	All	23–24

KPIs and risks

KPI	Target
Months loaded	6 (July–December 2025, 3,554,943 flights)
Pipeline run time for one month	Under 30 minutes on a 16 GB laptop
Scheduled runs that succeed	95% or more
Critical data-quality checks passing	100% on Gold
Re-running a month	Same row count, no duplicates
Dashboard page load	Under 5 seconds
Prediction API response	Under 500 ms
Delay model ROC-AUC on the December test month	0.70 or higher, and above the baseline
Assistant correct answers on 30-question test set	80% or higher

Risk	Mitigation
Silver layer is late and blocks the team	Dashboard and ML start on the July sample; Spark work starts first in week 16
Laptops lack memory for the full stack	Run only needed containers; process one month at a time; optional Databricks Free Edition
BTS website unavailable	Raw files kept in Bronze after first download; all six months already saved locally
Model score looks too good	Leakage review; time-based split only
Assistant writes wrong SQL	Read-only user, SQL shown to the user, test set, example questions in the prompt
Schedule slips	Tiered scope: Stretch, then Planned features are dropped before the MVP

Mapping to course modules

Course module	Where it is applied
Prompt Engineering	Prompt and example questions for the text-to-SQL assistant
Intro to Data Engineering	Medallion architecture, incremental monthly loads
Programming Essentials	Python download and validation scripts, unit tests
SQL & Database Management	Star schema in PostgreSQL, views, KPI queries
Advanced Python for Data Engineering	Modular pipeline package, Great Expectations
Data Pipelines	Airflow DAGs: scheduling, retries, alerts
Big Data Processing	PySpark and Delta Lake on 3.55M rows, partitioned by month
Microsoft Azure Data Engineer Associate	Architecture mapped one-to-one to Azure services (Technology stack table); optional Databricks run
Deployment	Docker Compose, GitHub Actions
AI for Data Engineers	Delay-prediction model with MLflow; GenAI assistant
Capstone project	This proposal and its deliverables